

Tool implementation — rules.py

src/lab_billing/reconstruct/rules.py — git blob e97482658b6e — v1.4.0 (2026-09-22)

```
"""Constrained linkage rules (module A, v0.7).

Each rule carries an ID, input dependencies, action, conflict priority and
source rationale. Rules only propose candidate pairs within an org and a
source system; they never cross orgs and never link on payer/service tokens
alone. When candidates are ambiguous the pair is queued for review instead
of being forced.

Rule inventory:
R1 native_previous_id      - native explicit predecessor (v0.2)
R2 same_bill_id_versions   - same org + bill_id, distinct version_ids
R3 amount_date_window      - weak evidence: same org, same payer/service,
                             close amounts, close dates (uniqueness required)
"""
from __future__ import annotations

from dataclasses import dataclass, field
from datetime import datetime, timedelta
from typing import Any

RULES = {
    "R1_native_previous_id": {
        "inputs": ["bills.previous_record_id"],
        "action": "accept link between referenced predecessor and successor",
        "conflict_priority": 1,
        "source": "native explicit reference in source data (spec section 10.2)",
    },
    "R2_same_bill_id_versions": {
        "inputs": ["bills.bill_id", "bills.version_id", "bills.org_token"],
        "action": "chain distinct versions of the same native bill number",
        "conflict_priority": 2,
        "source": "version chains within a native bill number (spec section 10.3)",
    },
    "R3_amount_date_window": {
        "inputs": ["bills.source_system", "bills.payer_token", "bills.service_token",
                  "bills.billed_amount_minor", "bills.submitted_at"],
        "action": "candidate pair on amount/date pattern; accept only when the "
                  "candidate is unique, otherwise queue for review",
        "conflict_priority": 3,
        "source": "business references, date windows and amount patterns "
                  "(spec section 10.3); weak evidence by design",
    },
}

@dataclass
class CandidatePair:
    from_id: str
    to_id: str
    rule_id: str
    evidence: str
    score: float = 0.0
```

```

score_kind: str = "UNVALIDATED_SCORE_UNLESS_CALIBRATED"

def generate_r2_candidates(bills: list[dict]) -> list[CandidatePair]:
    """Version chains within the same (org, bill_id)."""
    groups: dict[tuple, list[dict]] = {}
    for b in bills:
        groups.setdefault((b["org_token"], b["source_system"], b["bill_id"]),
[]) .append(b)
    out: list[CandidatePair] = []
    for (org, sysn, bid), members in groups.items():
        if len(members) < 2:
            continue
        # multiple rows claiming the same version => ambiguous
        versions: dict[str, list[dict]] = {}
        for m in members:
            versions.setdefault(m["version_id"], []).append(m)
        if any(len(v) > 1 for v in versions.values()):
            for m in members:
                out.append(CandidatePair(
                    from_id=m["record_id"], to_id="",
                    rule_id="R2_same_bill_id_versions",
                    evidence=f"ambiguous version set for {bid}",
                    score=0.0))
            continue
        ordered = sorted(members, key=lambda m: _ver_key(m["version_id"]))
        for prev, nxt in zip(ordered, ordered[1:]):
            out.append(CandidatePair(
                from_id=prev["record_id"], to_id=nxt["record_id"],
                rule_id="R2_same_bill_id_versions",
                evidence=f"version chain {prev['version_id']}->{nxt['version_id']} of
{bid}",
                score=0.9))
    return out

def generate_r3_candidates(bills: list[dict], window_days: int = 7,
                           amount_tol: float = 0.08) -> list[CandidatePair]:
    """Weak-evidence candidates: same org/payer/service, close amounts and
    dates. Uniqueness is enforced by the caller."""
    candidates: list[CandidatePair] = []
    by_key: dict[tuple, list[dict]] = {}
    for b in bills:
        if not b.get("payer_token") or not b.get("service_token"):
            continue # R3 needs both signals; never the sole basis
        by_key.setdefault((b["org_token"], b["source_system"],
                           b["payer_token"], b["service_token"]), []).append(b)
    for key, members in by_key.items():
        if len(members) < 2:
            continue
        for i in range(len(members)):
            for j in range(i + 1, len(members)):
                a, b = members[i], members[j]
                amt_a = a["billed_amount_minor"]
                amt_b = b["billed_amount_minor"]
                if min(amt_a, amt_b) == 0:
                    continue
                if abs(amt_a - amt_b) / max(amt_a, amt_b) > amount_tol:

```

```

        continue
        if abs((a["submitted_at"] - b["submitted_at"]).total_seconds()) >
window_days * 86400:
            continue
            candidates.append(CandidatePair(
                from_id=a["record_id"], to_id=b["record_id"],
                rule_id="R3_amount_date_window",
                evidence=f"same payer/service, amounts {amt_a}/{amt_b}, "
                        f"dates within {window_days}d",
                score=0.55))
    return candidates

def resolve_r3(candidates: list[CandidatePair]) -> tuple[list[CandidatePair],
list[CandidatePair]]:
    """Accept a pair only when each side has a single candidate; otherwise
    the pair goes to review. Rejected pairs (no acceptance, no review) are
    returned separately."""
    degree: dict[str, list[CandidatePair]] = {}
    for c in candidates:
        degree.setdefault(c.from_id, []).append(c)
        degree.setdefault(c.to_id, []).append(c)
    accepted, review = [], []
    for c in candidates:
        if len(degree[c.from_id]) == 1 and len(degree[c.to_id]) == 1:
            accepted.append(c)
        else:
            review.append(c)
    return accepted, review

def _ver_key(version_id: str) -> tuple:
    try:
        return (0, int(version_id))
    except ValueError:
        return (1, version_id)

```